bonsai workspace cleaner

https://gemini.google.com/app/57a91d966cc94e28

User prompt: I'm planning to build a tool to clean my workspace. my intent is to make it a cli to recursive look up from a directory and detect git repositories, so I it can discover all its worktrees. the tool must list them all to me, informing the their names, the repository, the disk space they spend and the last changed time. I must be able to select them for deletion. which name would you choose for a tool like that?

Response: Here are several name directions for a CLI tool focused on discovering, measuring, and pruning Git worktrees:

Snappy & CLI-Native (Short & Easy to Type)

- **`wt-prune`** / **`git-prune-wt`** — Straightforward, blends right into standard `git` naming conventions. If you name the binary `git-wtclean`, users can run it directly as `git wtclean`.
- **`treepruner`** — Clean and direct; immediately tells you what it prunes.
- **`arbor`** — Short for arboreal/tree care; carries an effortless UNIX feel.
- **`feller`** — A lumberjack who cuts down trees; punchy single-word binary.

Clean & Descriptive

- **`worktree-sweep`** / **`wtsweep`** — Conveys both the recursive scan and the cleanup aspect.
- **`git-dust`** — Evokes cleaning up accumulated workspace debris while staying very brief.
- **`branchtrim`** / **`wtrim`** — Emphasizes slimming down disk footprint without sounding destructive.

Playful & Metaphorical

- **`bonsai`** — Pruning away overgrown branches and worktrees to keep your local workspace compact and tidy.
- **`lopper`** — Named after heavy-duty pruning shears; fun to type (`lopper --all`).
- **`defoliate`** — Slightly more aggressive, perfect if the primary satisfaction is reclaiming gigabytes of disk space.

Top Pick

`bonsai` (or **`git-bonsai`**) It nails the exact mental model: you aren't destroying your main repository, you're carefully pruning overgrown worktree branches to keep your workspace lean and orderly.

If you prefer pure UNIX minimalism where keystroke efficiency is king, go with **`wtsweep`** or **`git-wtclean`**.

User prompt: Draft a CLI specification and terminal UI layout for this Git worktree cleaner tool.

Response: Here is a full CLI and Terminal UI (TUI) specification for **`bonsai`** (`git-bonsai`).

1. CLI Command Specification

Synopsis

```
bonsai [OPTIONS] [PATH]
```

If `PATH` is omitted, `bonsai` defaults to scanning the current working directory (`.`).

Command-Line Arguments & Flags

Flag	Short	Default	Description
`--path <DIR>`	`-p`	`.`	Root directory to begin the recursive scan.
`--depth <N>`	`-d`	`5`	Maximum directory recursion depth for finding repos.
`--min-age <DAYS>`	`-a`	`0`	Filter worktrees older than N days of inactivity.
`--min-size <SIZE>`	`-s`	`0`	Filter worktrees larger than size (e.g., `500M`, `2G`).
`--dry-run`	`-n`	`false`	Preview files and space that would be deleted.
`--force`	`-f`	`false`	Skip dirty-state confirmation checks (uncommitted changes).
`--plain`		`false`	Output plain tabular text (no TUI; useful for scripting/piping).
`--json`		`false`	Dump discovered worktree metadata as structured JSON.
`--help`	`-h`		Print usage information.
`--version`	`-v`		Print application version.

2. Terminal UI (TUI) Design

The TUI provides a scannable table view, dirty-tree safety indicators, and interactive multi-selection.

Main View (Scan & Selection)

```
bonsai v0.1.0 ── Scanning: ~/Projects ──────────────────────────────────────
Found 8 worktrees across 3 repositories · Reclaimable: 14.8 GB

  SELECT   REPOSITORY       WORKTREE / BRANCH        DISK       MODIFIED     STATUS
──────────────────────────────────────────────────────────────────────────────
[ ]        api-gateway      main (bare root)         2.1 GB     2 hours ago  clean
[x]   ❯    api-gateway      feat/oauth2              1.4 GB     18 days ago  clean
[x]        api-gateway      fix/jwt-expiry           1.1 GB     42 days ago  clean
[ ]        frontend-web     main                     3.8 GB     yesterday    clean
[x]        frontend-web     exp/tailwind-v4          2.9 GB     3 weeks ago  DIRTY
[ ]        data-pipeline    main                     1.2 GB     5 days ago   clean
[x]        data-pipeline    scratch/adhoc-etl        1.8 GB     2 months ago clean
[ ]        data-pipeline    feat/kafka-sink          1.5 GB     1 week ago   clean
──────────────────────────────────────────────────────────────────────────────
Selected: 3 worktrees (6.2 GB clean, 2.9 GB dirty)

[Space] Toggle  [a] Select All  [d] Delete (4)  [/] Filter  [q] Quit
```

Confirmation & Safety Modal (Triggered on `d`)

When dirty worktrees (uncommitted edits or untracked files) are selected, the tool intercepts before calling `git worktree remove`:

```
┌──────────────────────────────── Confirm Deletion ────────────────────────────┐
│                                                                              │
│  You are about to delete 4 worktrees to reclaim 9.1 GB:                      │
│                                                                              │
│   • api-gateway: feat/oauth2 (1.4 GB)                                        │
│   • api-gateway: fix/jwt-expiry (1.1 GB)                                     │
│   • data-pipeline: scratch/adhoc-etl (1.8 GB)                                │
│   ⚠ frontend-web: exp/tailwind-v4 (2.9 GB) [HAS UNCOMMITTED CHANGES]         │
│                                                                              │
│  Warning: Uncommitted changes will be PERMANENTLY lost.                      │
│                                                                              │
│  Type 'DELETE' to confirm, or press [Esc] to abort:                          │
```

```
> DELETE_
```

3. Keyboard Navigation

- `↑` / `k`, `↓` / `j` — Navigate rows.
- `Space` — Toggle selection on focused item.
- `a` — Invert / toggle all.
- `/` — Quick filter (filter by branch name or repo name).
- `o` — Cycle sort order (`size` → `modified` → `repo`).
- `d` — Trigger deletion sequence for selected items.
- `Esc` / `q` — Cancel modal or exit.

4. Backend Engine & Safety Rules

1. **Discovery Loop:**
   - Scan recursively for directories containing `.git` (folders or pointer files).
   - Run `git worktree list --porcelain` to get canonical paths, head commits, and branch names without parsing raw disk folders manually.
2. **Disk & Timestamp Calculation:**
   - **Size:** Calculate recursive disk usage (`du -sh` equivalent) of the worktree directory. Exclude hardlinks shared across identical blocks where applicable.
   - **Last Changed:** Inspect the highest `mtime` among the worktree root, latest commit timestamp via `git log -1 --format=%ct`, and recent file changes.
3. **Safety Invariants:**
   - **Protected Main/Bare Roots:** Prevent accidental selection of the primary repository root (`git rev-parse --is-bare-repository` or worktree indexed at position `0`).
   - **Dirty Check:** Run `git status --porcelain` in each target worktree before deletion. If dirty, require explicit `--force` or interactive confirmation override.
   - **Clean Removal:** Execute `git worktree remove <path>` followed by `git worktree prune` to ensure Git's administrative metadata in `.git/worktrees/` stays clean.

User prompt: which programming language would you use for building it? I'm thinking on using something able to work natively, such as kotlin, but I want opinions about it. I don't want this tool to depend many things, I.E., it must be easier to install.

Response: For a tool where **zero dependencies, effortless installation, and native execution** are top priorities, **Go** or **Rust** are significantly better suited than Kotlin Native.

The Reality of Kotlin Native for CLIs

While Kotlin/Native compiles to a standalone binary without needing the JVM:

- **Ecosystem Friction:** Most mature Kotlin CLI and TUI libraries (like Mordant or Clikt) are primarily designed for the JVM. Kotlin/Native's POSIX and terminal bindings are much less mature, meaning you will end up writing low-level C interop or fighting compiler quirks.
- **Toolchain Weight:** Kotlin/Native compile times and binary sizes are relatively heavy for small system utilities.

- **Verdict on Kotlin:** If you already write Kotlin daily and want to stick to familiar syntax, it is technically possible. However, building an interactive, multi-platform TUI with fast filesystem scanning is an uphill battle compared to languages built specifically for this niche.

Top Recommendation: Go

If you want to build and ship the tool with minimal friction, **Go is the sweet spot.**

- **Zero-Dependency Distribution:** Compiles to a single static binary with no shared library dependencies (`CGO_ENABLED=0`). Users can install it via a single `curl` script, `go install`, `brew`, or by downloading a pre-built binary release from GitHub.
- **The Charm Ecosystem:** Go has the premier terminal UI ecosystem today—specifically **Bubble Tea** (an Elm-architecture TUI framework), **Lip Gloss** (styling/layouts), and **Bubbles** (premade components like tables, spinners, and text inputs). Creating modern, polished TUIs in Go is faster than in almost any other language.
- **Fast Filesystem & Git Operations:** Go's standard library handles filesystem walks (`os.DirFS`, `filepath.WalkDir`) cleanly, and running subcommands (`os/exec` for `git worktree list --porcelain`) has negligible overhead.

The Alternative: Rust

If raw scanning throughput and strict safety guarantees matter most:

- **Maximum Performance:** Rust is the fastest option for scanning huge file trees and computing recursive disk sizes, especially using crates like `rayon` (data parallelism) or `ignore` (the engine behind `ripgrep` for fast directory traversal).
- **Top-Tier TUI Tooling: Ratatui** (formerly `tui-rs`) is the industry standard for production-grade terminal apps (powering tools like `bottom` and `gitui`).
- **Distribution:** Rust produces tiny, optimized static binaries (easily packaged via `cargo install`, `brew`, or `cargo-binstall`).
- **Tradeoff:** A steeper learning curve, more boilerplate around memory/lifetimes, and longer compile times during development compared to Go.

Language Comparison

Factor	Go	Rust	Kotlin (Native)
Installation Friction	Zero (single static binary)	Zero (single static binary)	Low (native binary), but large artifact
TUI Ecosystem	**Best** (`bubbletea`, `lipgloss`)	**Excellent** (`ratatui`, `crossterm`)	Minimal / experimental
Filesystem / Size Calc Speed	Very fast	Blazing fast (multi-threaded)	Moderate
Development Speed	Fastest	Slower (stricter compiler)	Moderate
Git Interop	Standard `os/exec` or `go-git`	`std::process` or `git2-rs`	Low-level C bindings or process calls

Recommendation

Choose **Go with Bubble Tea**. It yields a friction-free, single-binary distribution for macOS, Linux, and Windows, while giving you the quickest path to a clean, keyboard-driven terminal UI without wrestling with toolchains or complex borrow-checker semantics.

User prompt: Are there open source tools with the same purpose out there? I want to do some benchmarking analysis before proceeding

Response: There are several open-source tools that touch Git worktrees, but **almost none do the exact combination of recursive workspace discovery + disk usage calculation + bulk pruning.**

Existing tools fall into three distinct categories. Reviewing them shows where they stop short and where your tool has a clear competitive edge.

Category 1: Single-Repository Worktree Workflow Managers

These are meant to be used *while working* inside a single repository (creating branches, switching directories, running hooks).

- **`worktrunk` (`wt`)** (Rust)
  - **Focus:** Automating worktree creation, branch switching, and lifecycle for parallel development and AI coding agents.
  - **How cleanup works:** Offers `wt remove` or branch merging workflows.
  - **Limitation:** Repository-scoped. It operates on the current repo and does not recursively scan `~/Projects` across multiple unrelated codebases, nor does it calculate recursive disk footprints.
- **`grove`** (Go)
  - **Focus:** Structuring bare clones and worktrees cleanly.
  - **How cleanup works:** Offers `grove prune` (prunes branches merged into `main` or older than a duration like `--older-than 30d`).
  - **Limitation:** Only manages trees belonging to a specific configured root project hierarchy. No disk footprint metrics and no interactive multi-select UI.
- **`wtp` (Worktree Plus)** (Go)
  - **Focus:** Fast worktree creation with setup hooks (`npm install`, copying `.env`).
  - **How cleanup works:** `wtp remove --with-branch`.
  - **Limitation:** Single-repo task runner; not an interactive disk reclamation tool.

Category 2: Worktree Dashboard TUIs

- **`gwm` / `gwm-cli` & `loom-tui`** (Rust)
  - **Focus:** Terminal dashboards for monitoring active branches, PR statuses, and switching between worktrees.
  - **How cleanup works:** Single-item removal (`x` or `delete`).
  - **Limitation:** Designed as workflow "command centers" rather than janitorial storage tools. They track Git status (commits, remote sync), not disk volume consumed by build artifacts (`node_modules`, `target/`, `.venv`).

Category 3: General Disk Cleaners (Non-Worktree)

- **`kondo`** (Rust)
  - **Focus:** Recursively scans directories to detect and delete build artifacts (`node_modules`, `target`, `build`, etc.) to reclaim space.
  - **Limitation:** Completely agnostic to Git worktrees. It deletes build folders inside a worktree, but leaves the worktree registration, Git pointer, and branch intact.
- **`dust` / `ncdu` / `gdu`**
  - **Focus:** Visualizing large directories on disk.
  - **Limitation:** Generic filesystem navigators. They don't know what a `.git` link is or how to run `git worktree remove` safely.

Benchmarking Summary & The Gap

Feature	`worktrunk` / `grove`	`loom-tui` / `gwm`	`kondo`	Your Proposed Tool
Scope	Single project root	Single project / session	Recursive workspace	**Recursive workspace**
Disk Space Reporting	No	No	Yes (build artifacts only)	**Yes (full worktree size)**
Last Active / mtime	Git commit time only	Branch activity	Directory mtime	**Commit + file mtime**
Interactive Selection	Minimal (fuzzy pick)	Dashboard list	TUI available	**Multi-select TUI**
Safe Git Removal	Yes (`git worktree remove`)	Yes	No (raw file deletion)	**Yes (`git worktree remove`)**

The Opportunity

Your project fills a distinct gap: **a storage-focused janitor for developers using worktrees across many projects.**

Most developers run into the exact pain point you described: worktrees accumulate across 5–10 different repositories over months, each holding hidden gigabytes of dependencies (`node_modules`, `build/`, `venv`), but standard tools only look at one repo at a time. Building this as a dedicated CLI/TUI tool gives it immediate differentiation.

Sources

1. Worktrunk is a CLI for Git worktree management, designed ... - GitHub
2. GitHub - captainsafia/grove: CLI tool for managing git worktree
3. GitHub - satococoa/wtp: A powerful Git worktree CLI tool with
4. loom-tui — Rust utility // Lib.rs
5. Git Worktrees | Ralph TUI
